Android - Unit 4

Navigation

Android - Unit 5

Connect to the internet
Coroutines

User Interfaces 2

Unit 4: UI architecture

1. Architecture Components (vorige module)
2. Navigation in Jetpack Compose
3. Adapt for different screen sizes (slaan we over)



Navigation concepten



- Elke **destination** (composable) waar je heen kan navigeren heeft een unieke naam (**route**) **-> String**
- **NavGraph**: de verschillende routes waarheen je kan navigeren in je toepassing
- **NavController**: voert de navigatie naar een nieuw scherm uit via **navController**.navigate(**route**)
- **NavHost**: de composable container die de huidige **destination** bevat
    - Constructor parameters: **NavController** en start **destination**

Navigation Code along: CupCake App 

- We gebruiken een Scaffold met een topNavBar
- De Scaffold **content** bevat de `NavHost`

Navigation code structure



maak navController

```
fun AppScreen(navController: NavHostController = rememberNavController()) {
    NavHost(modifier = modifier, navController = navController,
      startDestination = "profile"
    ) {
       composable("profile") {
         ProfileScreen(
            onNavigateToFriends = {
                    navController.navigate("friendsList")
         })
       }
       composable("friendslist") { FriendsListScreen(/*...*/) }
       composable("portfolio") {ProjectsScreen(/*...*/) }
       composable("badges") { BadgesScreen(/*...*/) }
            /*...*/
        }
}
```

route

verbind navController met één NavHost component

composable() is een extension functie van NavGraphBuilder.
Roep deze aan voor elke route en geef als inhoud het te tonen scherm.
Al deze combinaties samen vormen de NavGraph.

Navigation code structure



```
fun AppScreen(navController: NavHostController = rememberNavController()) {
    NavHost(modifier = modifier, navController = navController,
      startDestination = "profile"
    ) {
       composable("profile") {
         ProfileScreen(
            onNavigateToFriends = {
                    navController.navigate("friendslist")
         })
       }
       composable("friendslist") { FriendsListScreen(/*...*/) }
       composable("portfolio") {ProjectsScreen(/*...*/) }
       composable("badges") { BadgesScreen(/*...*/) }
            /*...*/
        }
}
```

Destination `"profile"` *toont het* `ProfileScreen`*.*
Design tip: door een callback mee te geven aan `ProfileScreen` *met een navigatie naar het scherm dat erna zal komen (*`"friendslist"`*) kan je alle navigatie links/logica hier centraliseren*

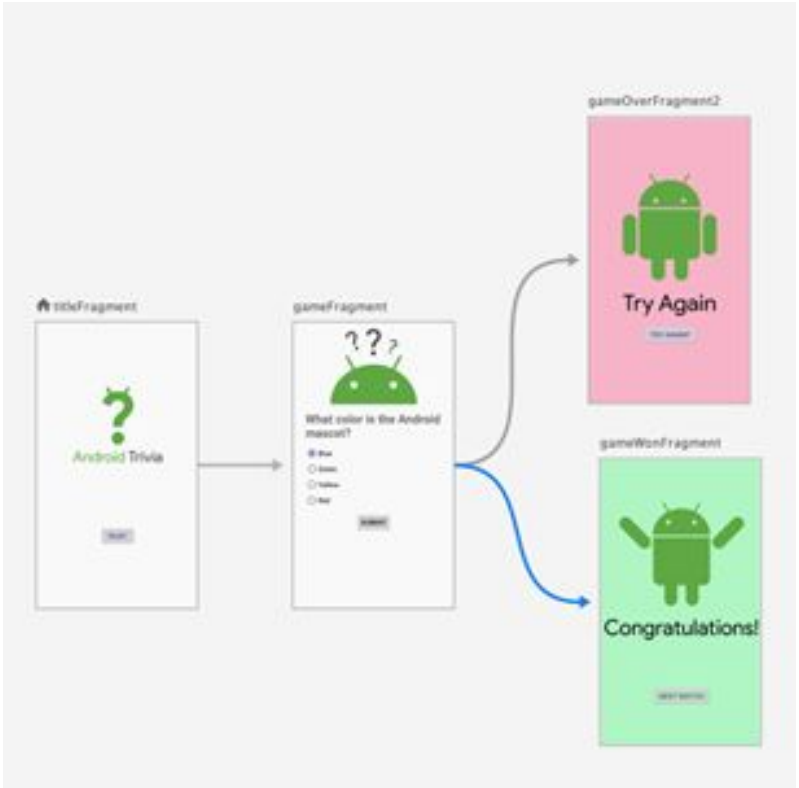
Design tip: als je een enum gebruikt voor je routes vermijd je tikfouten

Conditionele route



Je kan de route waar je heen navigeert laten afhangen van de applicatie logica

```
navController.navigate( route )
```

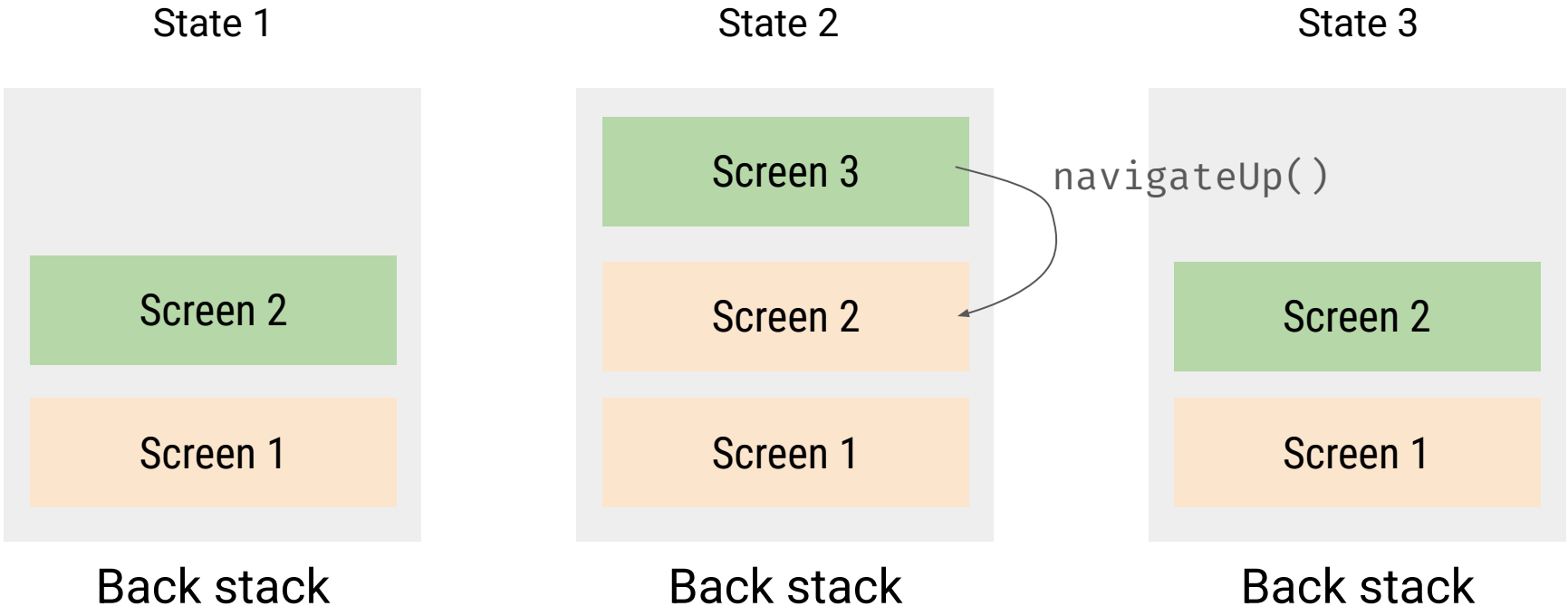


Move up



In de “BackStack” houdt de **`navController`** een historiek van de schermen die je bezocht.
Met de android back button of **`navController.navigateUp()`** kan je terug naar het vorige scherm.

`navigateUp()` gaat nooit verder dan screen 1



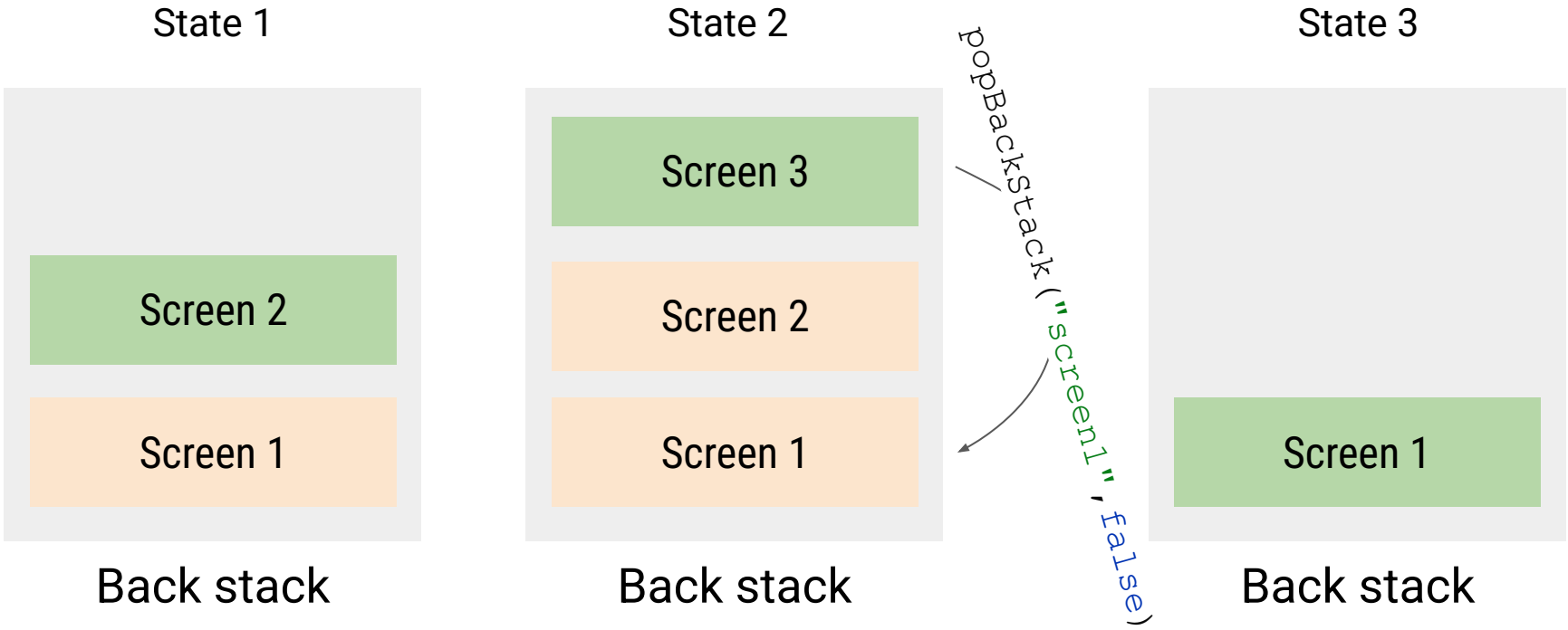
Move back



- Om meerdere schermen terug te gaan gebruik je

```
navController.popBackStack( route , inclusive )
```

  - Zonder parameters navigeert `popBackStack` één scherm omhoog
  - navigate back voorbij screen 1 verlaat de app



Complexere routes

Voorbeeld code



- Je kan complexere routes meegeven aan `composable()` functie met "/" én parameters meesturen met {}
  - Niet in Google cursus
  - Voorbeeld: **navController.navigate("profile/3")**
  - Haal waarden op via backStackEntry.arguments

```
NavHost(
    navController = navController,
    startDestination = "profile/{userId}"
) {
    composable("profile/{userId}") { backStackEntry ->

        val profileId = backStackEntry.arguments?.getString("userId")

        profileId?.let { Log.d("Route Argument", it) }

    }
}
```

Path parameter zoals bij REST URLs

Je haalt de waarde van de parameter op.

LET OP! Parameter kan null zijn..

Supported_argument_types

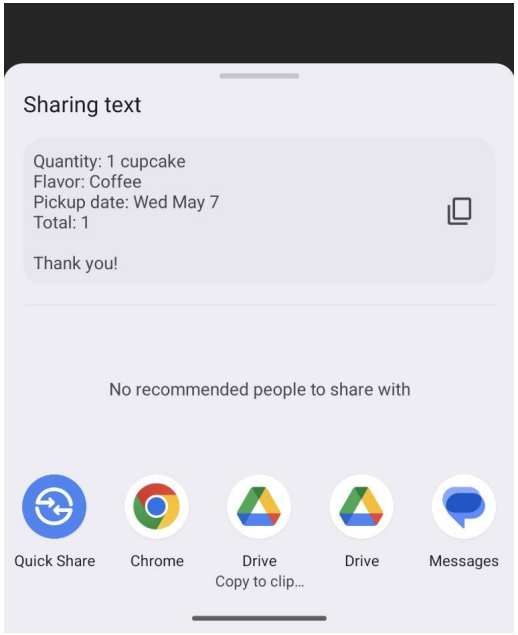
Intents



Met behulp van Intents kun je ook naar een andere app navigeren.

Bv: **“ShareSheet”**: een onderdeel van de gebruikersinterface opties voor delen

```
val intent = Intent(Intent.ACTION_SEND).apply {
    type = "text/plain"
    putExtra(Intent.EXTRA_SUBJECT, subject)
    putExtra(Intent.EXTRA_TEXT, summary)
}
context.startActivity(
    Intent.createChooser(
        intent,
        "Share this text"
    )
)
```



Intents



Soorten Intents:

- Impliciet: je specificeert het type app en de gebruiker kiest de app zelf
- Expliciet: je geeft precies aan naar welke app de gebruiker gaat (niet in het codelab)

Oefening



- Practice: Add navigation (lunchtray)



(codeTest the Cupcake app)

Unit 5: Connect to the internet

Coroutines

1. Get data from the internet (co-routines)
    1. Connect to the internet
    2. Introduction to Coroutines in Kotlin
    3. Introduction to Coroutines in Android

       ======= volgende Canvas module ==========
    4. Introduction to HTTP/REST
    5. Get data from the internet
2. Load and display images from the internet



Coroutines: Concepten



- Concepten: zie Android Compose cursus
- **coroutine**: context waarbinnen je code parallel (asynchroon) *kan* uitvoeren.

`build.gradle.kts` dependency:

```
implementation("org.jetbrains.kotlinx:kotlinx-coroutines-core:1.x.x")
```

Coroutines: Suspend functies



- `suspend` **modifier**: functie die gepauzeerd / hernomen kan worden (en dus parallel uitgevoerd kan worden). Kan je enkel aanroepen in:
  - Een CoroutineContext
  - Een andere suspend function (die wacht op zijn uitvoering)

Maakt een nieuwe `coroutine`*, die de {body}* ***sequentieel*** *uitvoert. {body } eindigt pas als elke suspend functie gedaan is.*

```
fun main() {
    runBlocking {
        println("Weather forecast")
        delay(1000)
        println("30°C")
    }
}
```

```
import kotlinx.coroutines.delay;

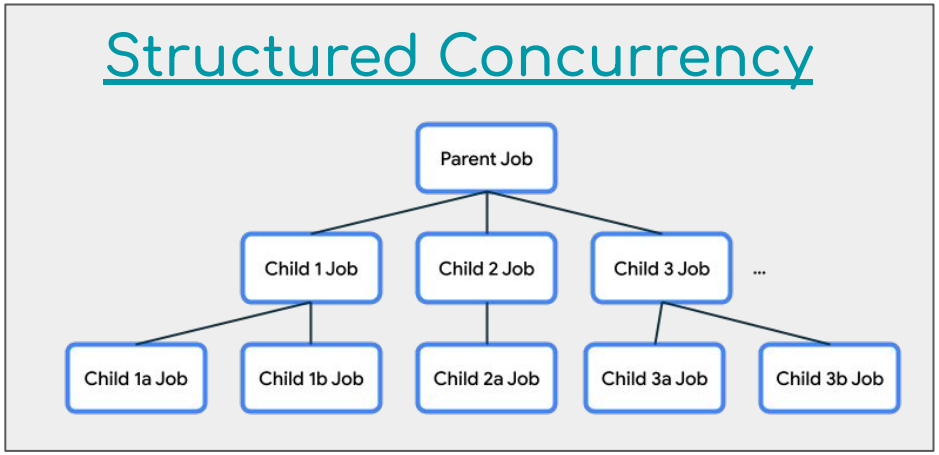
suspend fun main() {
    println("Weather forecast")
    delay( timeMillis: 1000)
    println("Sunny")
}
```

`suspend fun main` *heeft hetzelfde effect als de runblocking code hiernaast*

CoroutineScope



- Structured concurrency: laat je coroutines samen beheren
  - eindigt pas als alle child `jobs` beëindigd zijn
  - cancel van de *scope* → cancelt alle children
  - Cancel van een *child* `job.cancel()` cancelt de *parent* <u>niet</u>
  - Een (child) `job` met een **exception** cancelt de parent met die exception



CoroutineScope



- een CoroutineScope heeft een **`CoroutineContext`**
- De **`CoroutineContext`** bevat de **Dispatcher**, de naam, Job, Parent Job ...

Kotlin Coroutine builders: sequentieel



- `runBlocking{body}` maakt een root coroutine (met CoroutineContext) binnen synchrone code (vb main())
  - `this` in `runBlocking{body}` verwijst naar de nieuwe `CoroutineScope`
  - Voert suspend functies **sequentieel** uit
- `coroutineScope{body}` werkt zoals `runBlocking`, maar maakt een child binnen een **bestaande** parent `CoroutineScope`
  - handig om te wachten op een taak bestaande uit een groep asynchrone children

Kotlin Coroutine builders: parallel



- Enkele methodes die coroutines **parallel** starten binnen een bestaande `CoroutineScope` :
  - `launch{`*`body`*`}:Job`
    - `Job`: coroutine run state (voor cancel...)
    - "Fire and forget"
  - `async{`*`body`*`}:Deferred<T>`
    - Heeft return value (zoals JS async functie)
    - parallelle uitvoering die *resultaat* teruggeeft als `Deferred`
      - Resultaat is opvraagbaar met `.await()`
      - `Deferred` is een subklasse van `Job`

Code voorbeeld uit de android cursus

Dispatchers



- De dispatcher bepaalt in welke Thread de coroutine runt
- Een child coroutine erft de context en dispatcher van zijn parent, als hij geen nieuwe dispatcher specificeert.

```
launch (Dispatchers.Default) {
    println(Thread.currentThread().name)
}
// DefaultDispatcher-worker-1
```

Thread name opzoeken met java library

Built-in Dispatchers



- **Dispatchers.Main** - Mainthread van Android. Deze moet alleen worden gebruikt voor interactie met de UI en het uitvoeren van snel werk.
    - Voorbeelden zijn het aanroepen van suspend functies,
    - uitvoeren van Android UI framework operaties
    - updaten van LiveData of StateFlow objecten.
    - **Run geen code die lang duurt op de main thread**
- **Dispatchers.IO** - Deze dispatcher is geoptimaliseerd om schijf of netwerk I/O uit te voeren buiten de Main thread.
    - gebruik van de Room Database component,
    - lezen van of schrijven naar bestanden
    - uitvoeren van netwerk operaties.
- **Dispatchers.Default** - CPU-intensief werk buiten de Main thread.
    - Voorbeelden zijn het sorteren van een lijst en het parsen van JSON.

Voorbeeld [Dispatcher Playground demo]

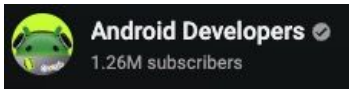
Coroutines in Kotlin: conclusions



Launch, *completion*, *cancellation*, and *failure* zijn vier veel voorkomende operaties in coroutine jobs.

- **Launch**: Start een coroutine in een scope, geeft een Job terug om de levenscyclus te managen.
- **Completion**: De job is niet compleet totdat zijn kind-jobs compleet zijn.
- **Cancellation**: Propageert naar beneden. Als een coroutine wordt geannuleerd, worden ook de coroutines van de kinderen geannuleerd.
- **Failure/Exception**: Propageert naar boven. Als een coroutine een exception throwt, dan annuleert hij al zijn kinderen, annuleert zichzelf en propageert de uitzondering naar de parent. Dit gaat door totdat de exception is opgevangen en afgehandeld.

Coroutines 101 video:



https://www.youtube.com/watch?v=ZTDXo0-SKuU&t=2s

Coroutines in Android



- Lifecycle aware scopes
  - `LifeCycleScope`: coroutine scope wordt gelinkt aan de lifecycle van een component (voorbeeld Activity).
    - In een Composable kan je coroutines maken met
      - `LaunchedEffect {}` **composable** maakt coroutine die wordt gestart als ze in de parent composable komen en gecanceled als het LaunchedEffect de composable verlaat
      - `SideEffect {}` is een variant waarbij de coroutine pas loopt op het einde van de Composable
  - `ViewModelScope:` Coroutines die je start op `viewModelScope.launch {}` in een ViewModel worden gecanceld als het viewModel vernietigd wordt

Coroutines in Android



- ~~Coroutine unit tests~~